

Tech Sales Dataset Overview & Data Dictionary

Dataset Summary

This dataset contains transactional sales records for a technology-focused retail and wholesale business. The data captures customer orders across multiple product categories, customer segments, and U.S. states.

High-Level Characteristics

Attribute	Value
Total Records	4,976
Total Columns	13
Date Range	2017-09-01 to 2019-08-30
Geographic Coverage	50 U.S. states
Unique Products	65
Order Types	Retail, Wholesale
Customer Types	Business, Individual
Approximate Total Revenue	\$6.90M

Dataset Purpose

The dataset appears designed for:

- Sales analytics and reporting
- Revenue trend analysis
- Customer segmentation
- Product performance analysis
- Retail vs. wholesale comparisons
- Geographic sales analysis
- Discount and pricing analysis
- Forecasting and business intelligence exercises

This is a classic transactional sales dataset suitable for:

- SQL practice
- Tableau/Power BI dashboards
- Python or R analytics projects
- Machine learning forecasting exercises

- KPI reporting
 - Educational demonstrations
-

Dataset Structure

Each row represents a single customer order line item.

The dataset combines:

- Customer information
 - Product information
 - Order metadata
 - Pricing information
 - Revenue calculations
-

Detailed Data Dictionary

1. OrderNum

Property	Value
Description	Unique identifier for each sales order
Data Type	Integer
Example	1100934
Business Meaning	Tracks and references individual customer orders
Analytical Use	Order-level aggregation, transaction counting, joins

Notes

- Appears sequentially generated
 - Likely primary key candidate
 - Useful for detecting duplicate orders
-

2. OrderDate

Property	Value
Description	Date the order was placed

Property	Value
Data Type	Date (stored as text in raw CSV)
Example	9/01/2017
Business Meaning	Transaction timestamp
Analytical Use	Time-series analysis, seasonality, forecasting

Notes

- Should be converted to a true datetime field during preprocessing
- Supports monthly, quarterly, and yearly analysis
- Can be used for trend and growth calculations

Recommended Derived Fields

- Year
- Quarter
- Month
- Week
- Day of Week
- Fiscal Period

3. OrderType

Property	Value
Description	Indicates sales channel or purchasing arrangement
Data Type	Categorical/Text
Possible Values	Retail , Wholesale
Example	Wholesale
Analytical Use	Channel analysis, margin analysis, segmentation

Business Interpretation

- **Retail** likely represents smaller direct customer purchases
- **Wholesale** likely represents bulk or reseller transactions

Potential Analysis

- Average order size by channel
- Revenue contribution by order type
- Discount comparison by channel

4. CustomerType

Property	Value
Description	Customer classification
Data Type	Categorical/Text
Possible Values	Business, Individual
Example	Business
Analytical Use	Customer segmentation and behavioral analysis

Business Interpretation

- **Business** customers may purchase in higher volume
- **Individual** customers likely represent end consumers

Potential Analysis

- Customer lifetime value
- Revenue by customer segment
- Purchase frequency patterns

5. CustName

Property	Value
Description	Customer or organization name
Data Type	Text
Example	Gusikowski Group
Analytical Use	Customer-level reporting and grouping

Notes

- Contains both company and personal-style names
- Could be anonymized or synthetic sample data
- Useful for identifying repeat customers

Potential Analysis

- Top customers by revenue
- Repeat purchase analysis

- Customer concentration analysis
-

6. CustState

Property	Value
Description	U.S. state associated with customer
Data Type	Text
Example	Texas
Analytical Use	Geographic sales analysis

Notes

- Includes all 50 U.S. states
- Supports regional sales mapping

Potential Analysis

- Revenue by state
 - Product preferences by region
 - Regional customer concentration
-

7. ProdCategory

Property	Value
Description	High-level product grouping
Data Type	Categorical/Text
Example	Robot Kits
Analytical Use	Product mix and category performance analysis

Example Categories

- Robot Kits
- Drone Kits
- eBooks
- Training Videos
- Blueprints

Potential Analysis

- Best-performing categories
 - Revenue share by category
 - Seasonal category demand
-

8. ProdNumber

Property	Value
Description	Product SKU or product identifier
Data Type	Text
Example	RK602
Analytical Use	Product-level tracking and joins

Notes

- Likely unique per product
- Useful for inventory or product catalog joins

Potential Analysis

- Product-level profitability
 - SKU sales rankings
 - Product demand forecasting
-

9. ProdName

Property	Value
Description	Human-readable product name
Data Type	Text
Example	BYOR-1000
Analytical Use	Product reporting and descriptive analysis

Notes

- Product names suggest robotics and drone technology themes
- May contain branded or model identifiers

Potential Analysis

- Top-selling products
 - Product bundling patterns
 - Product popularity trends
-

10. Quantity

Property	Value
Description	Number of units purchased
Data Type	Integer
Example	10
Analytical Use	Volume analysis and demand measurement

Notes

- Critical for revenue calculations
- Useful for identifying bulk purchases

Potential Analysis

- Average order quantity
 - Bulk order detection
 - Inventory demand estimation
-

11. Price

Property	Value
Description	Unit price of product
Data Type	Currency stored as text
Example	\$89.00
Analytical Use	Pricing and revenue analysis

Important Data Cleaning Note

This field contains:

- Dollar signs (\$)
- Text formatting

- String data type in raw CSV

Recommended Transformation

Convert to numeric decimal format.

Example:

Raw Value	Cleaned Value
\$89.00	89.00

Potential Analysis

- Price elasticity
- Average selling price
- Product pricing comparisons

12. Discount

Property	Value
Description	Discount amount applied to order
Data Type	Currency stored as text
Example	\$1.80
Analytical Use	Promotional effectiveness and margin analysis

Important Data Cleaning Note

This field should be converted from string to numeric.

Business Interpretation

Represents either:

- Total line-item discount amount, or
- Per-unit discount value

Further validation may be required.

Potential Analysis

- Discount effectiveness
- Margin impact

- Discount frequency by customer type
-

13. Order Total

Property	Value
Description	Final total value of the order line
Data Type	Currency stored as text
Example	\$178.00
Analytical Use	Revenue analysis and KPI reporting

Important Data Cleaning Note

This field should be converted to numeric format.

Likely Formula

Order Total = (Quantity × Price) - Discount

Potential Analysis

- Revenue trends
 - Average order value
 - Top-performing customers/products
 - Forecasting
-

Data Quality Assessment

Strengths

- No obvious missing values
- Consistent transactional structure
- Good categorical organization
- Multi-dimensional analytical potential
- Reasonable business realism

Data Cleaning Requirements

1. Currency Conversion

The following columns should be converted from text to numeric:

- Price
- Discount
- Order Total

2. Date Parsing

OrderDate should be parsed into a standard datetime format.

3. Column Name Cleanup

Some columns contain leading/trailing spaces:

- Price
- Discount
- Order Total

Recommended cleanup:

```
# Example cleanup
[df.columns.str.strip()]
```

4. Validation Checks

Recommended validation rules:

- Ensure Quantity > 0
- Ensure prices are non-negative
- Validate total calculations
- Check for duplicate order IDs

In-Depth Data Cleaning Recommendations (Ranked by Priority)

Proper data cleaning is essential before performing analytics, dashboarding, forecasting, or machine learning. Below is a prioritized roadmap for preparing this dataset for production-quality analysis.

Priority 1 — Standardize Column Names

Why This Matters

Several columns contain leading/trailing spaces:

- Price
- Discount
- Order Total

This creates problems when:

- Writing SQL queries
- Referencing columns in Python
- Building dashboards
- Performing joins or transformations

Risks if Ignored

- KeyErrors in pandas
- Broken pipelines
- Inconsistent naming conventions
- Harder collaboration

Recommended Fix

```
# Remove whitespace from column names

df.columns = df.columns.str.strip()
```

Recommended Enhancement

Convert all columns to a standardized naming convention.

```
# Snake case formatting

df.columns = (
    df.columns
    .str.strip()
    .str.lower()
    .str.replace(' ', '_')
)
```

Example Result

Original	Cleaned
Order Total	order_total
CustState	cust_state

Priority 2 — Convert Currency Fields to Numeric

Why This Matters

The following fields are stored as text:

- Price
- Discount
- Order Total

Values contain:

- Dollar signs (\$)
- Commas
- String formatting

Risks if Ignored

- Mathematical operations fail
- Aggregations become incorrect
- Sorting behaves alphabetically instead of numerically
- Machine learning models cannot use the fields

Recommended Fix

```
currency_cols = ['Price', 'Discount', 'Order Total']

for col in currency_cols:
    df[col] = (
        df[col]
        .replace('[$,]', '', regex=True)
        .astype(float)
    )
```

Additional Validation

```
# Check for failed conversions  
  
print(df[currency_cols].isnull().sum())
```

Recommended Enhancement

Use decimal precision for financial calculations.

```
from decimal import Decimal
```

Especially important for:

- Financial reporting
- Tax calculations
- ERP integrations

Priority 3 — Parse and Validate Date Fields

Why This Matters

`OrderDate` is stored as text instead of a true datetime object.

Risks if Ignored

- Time-series analysis becomes impossible
- Forecasting pipelines fail
- Incorrect date sorting
- Inability to create rolling averages or trends

Recommended Fix

```
# Convert to datetime  
  
df['OrderDate'] = pd.to_datetime(df['OrderDate'])
```

Recommended Validation

```
# Check min/max dates
```

```
print(df['OrderDate'].min())
print(df['OrderDate'].max())
```

Additional Quality Checks

```
# Detect invalid dates

invalid_dates = df[df['OrderDate'].isnull()]
print(invalid_dates)
```

Recommended Derived Features

```
# Feature engineering

df['year'] = df['OrderDate'].dt.year
df['month'] = df['OrderDate'].dt.month
df['quarter'] = df['OrderDate'].dt.quarter
df['day_of_week'] = df['OrderDate'].dt.day_name()
```

These features are critical for:

- Seasonality analysis
- Forecasting
- Executive dashboards

Priority 4 — Validate Revenue Calculations

Why This Matters

The dataset likely follows this formula:

```
Order Total = (Quantity × Price) - Discount
```

However, datasets often contain:

- Rounding errors
- Incorrect discounts
- Data-entry mistakes
- Partial discounts

Risks if Ignored

- Financial reporting errors
- Incorrect KPI calculations
- Misleading dashboards

Recommended Validation

```
# Recalculate totals

df['calculated_total'] = (
    df['Quantity'] * df['Price']
) - df['Discount']

# Compare differences

df['difference'] = (
    df['Order Total'] - df['calculated_total']
).round(2)

# Find mismatches

mismatches = df[df['difference'] != 0]
print(mismatches.head())
```

Recommended Resolution Strategy

Situation	Recommendation
Minor rounding differences	Accept tolerance (± 0.01)
Large discrepancies	Investigate manually
Missing totals	Recalculate programmatically

Priority 5 — Detect Missing Values

Why This Matters

Missing data can bias:

- Revenue calculations
- Forecasting models
- Customer segmentation
- KPI reporting

Recommended Checks

```
# Missing value counts  
  
print(df.isnull().sum())
```

Recommended Strategy by Column Type

Column Type	Recommended Handling
IDs	Never allow nulls
Dates	Remove or infer
Categories	Fill with 'Unknown'
Numeric fields	Impute or remove

Example

```
# Fill categorical nulls  
  
df['CustomerType'] = df['CustomerType'].fillna('Unknown')
```

Priority 6 — Remove Duplicate Records

Why This Matters

Duplicate transactions can:

- Inflate revenue
- Distort customer counts
- Break forecasting models

Recommended Checks

```
# Detect duplicates  
  
duplicates = df[df.duplicated()]  
print(duplicates)
```

Order-Level Duplicate Detection

```
# Duplicate order numbers  
  
duplicate_orders = df[df.duplicated('OrderNum', keep=False)]
```

Recommended Resolution

Duplicate Type	Action
Exact duplicate rows	Remove
Same order with different products	Keep
Duplicate order IDs	Investigate

Priority 7 — Validate Numeric Fields

Why This Matters

Business datasets often contain impossible values.

Recommended Checks

Negative Prices

```
print(df[df['Price'] < 0])
```

Zero or Negative Quantities

```
print(df[df['Quantity'] <= 0])
```

Extreme Outliers

```
print(df['Order Total'].describe())
```

Outlier Detection Example

```
Q1 = df['Order Total'].quantile(0.25)  
Q3 = df['Order Total'].quantile(0.75)  
IQR = Q3 - Q1
```

```
outliers = df[
    (df['Order Total'] < Q1 - 1.5 * IQR) |
    (df['Order Total'] > Q3 + 1.5 * IQR)
]
```

Important Note

Outliers are not always errors.

Large wholesale orders may be legitimate.

Priority 8 — Standardize Categorical Values

Why This Matters

Categorical inconsistency causes fragmented analytics.

Example:

```
Business
business
BUSINESS
```

These become separate groups.

Recommended Fix

```
categorical_cols = [
    'OrderType',
    'CustomerType',
    'ProdCategory',
    'CustState'
]

for col in categorical_cols:
    df[col] = df[col].str.strip().str.title()
```

Additional Validation

```
for col in categorical_cols:
    print(df[col].unique())
```

Priority 9 — Optimize Data Types

Why This Matters

Efficient data types improve:

- Memory usage
- Query speed
- Dashboard performance
- ML pipeline efficiency

Recommended Optimization

```
# Convert categorical columns

categorical_cols = [
    'OrderType',
    'CustomerType',
    'ProdCategory',
    'CustState'
]

for col in categorical_cols:
    df[col] = df[col].astype('category')
```

Benefits

Benefit	Impact
Lower memory usage	Faster processing
Faster grouping	Better analytics
Smaller files	Easier deployment

Priority 10 — Create Data Validation Rules

Why This Matters

Validation rules prevent future bad data.

Recommended Rules

Field	Rule
Quantity	Must be > 0
Price	Must be $\geq$ 0
Discount	Must be $\geq$ 0
Order Total	Must match formula
OrderDate	Cannot be future date

Example Validation Pipeline

```
assert (df['Quantity'] > 0).all()
assert (df['Price'] >= 0).all()
assert (df['Discount'] >= 0).all()
```

Recommended Production Cleaning Workflow

Step-by-Step Pipeline

```
# 1. Load data
# 2. Clean column names
# 3. Parse dates
# 4. Convert currencies
# 5. Remove duplicates
# 6. Handle missing values
# 7. Validate totals
# 8. Standardize categories
# 9. Detect anomalies
# 10. Export cleaned dataset
```

Recommended Analytics Projects

Beginner

- Total sales by month
- Top-selling products
- Revenue by state
- Retail vs. wholesale sales

Intermediate

- Customer segmentation
- Basket size analysis
- Discount effectiveness
- Product category growth trends

Advanced

- Sales forecasting
 - Customer lifetime value modeling
 - Churn prediction
 - Product recommendation systems
 - Revenue anomaly detection
-

Suggested KPI Metrics

Revenue KPIs

- Total Revenue
- Monthly Revenue Growth
- Average Order Value
- Revenue by Product Category

Customer KPIs

- Unique Customers
- Revenue per Customer
- Repeat Purchase Rate
- Business vs Individual Revenue Share

Product KPIs

- Units Sold

- Top Products
 - Category Contribution
 - Average Selling Price
-

Example Business Questions This Dataset Can Answer

1. Which states generate the highest revenue?
 2. Which products are most popular among business customers?
 3. Are wholesale orders larger than retail orders?
 4. Which product categories grow fastest over time?
 5. How do discounts affect total sales?
 6. What are the peak sales months?
 7. Which customers contribute the most revenue?
 8. What is the average order value by customer type?
-

Python Data Exploration & Data Dictionary Script

```
import pandas as pd

# Load dataset
file_path = "Tech Sales Dataset.csv"
df = pd.read_csv(file_path)

# Display basic information
print("Dataset Shape:")
print(df.shape)

print("
Column Names:")
print(df.columns.tolist())

print("
Dataset Info:")
print(df.info())

print("
First 5 Rows:")
print(df.head())

# Clean column names
```

```

df.columns = df.columns.str.strip()

# Convert date column
df['OrderDate'] = pd.to_datetime(df['OrderDate'])

# Clean currency columns
currency_cols = ['Price', 'Discount', 'Order Total']

for col in currency_cols:
    df[col] = (
        df[col]
        .replace('[$,]', '', regex=True)
        .astype(float)
    )

# Generate summary statistics
print("
Summary Statistics:")
print(df.describe(include='all'))

# Create automated data dictionary

data_dictionary = pd.DataFrame({
    'Column Name': df.columns,
    'Data Type': df.dtypes.astype(str).values,
    'Missing Values': df.isnull().sum().values,
    'Unique Values': [df[col].nunique() for col in df.columns],
    'Example Value': [df[col].dropna().iloc[0] for col in df.columns]
})

print("
Data Dictionary:")
print(data_dictionary)

# Save cleaned dataset
cleaned_file = 'cleaned_tech_sales_dataset.csv'
df.to_csv(cleaned_file, index=False)

# Save data dictionary
output_dictionary = 'tech_sales_data_dictionary.csv'
data_dictionary.to_csv(output_dictionary, index=False)

print(f"
Cleaned dataset saved as: {cleaned_file}")
print(f"Data dictionary saved as: {output_dictionary}")

```

Example Python Analytics

Total Revenue

```
total_revenue = df['Order Total'].sum()
print(f"Total Revenue: ${total_revenue:,.2f}")
```

Top 10 Products by Revenue

```
top_products = (
    df.groupby('ProdName')['Order Total']
      .sum()
      .sort_values(ascending=False)
      .head(10)
)

print(top_products)
```

Monthly Sales Trend

```
monthly_sales = (
    df.groupby(df['OrderDate'].dt.to_period('M'))['Order Total']
      .sum()
)

print(monthly_sales)
```

Revenue by State

```
state_sales = (
    df.groupby('CustState')['Order Total']
      .sum()
      .sort_values(ascending=False)
)

print(state_sales.head(10))
```

Retail vs Wholesale Comparison

```
channel_sales = (  
    df.groupby('OrderType')['Order Total']  
    .agg(['sum', 'mean', 'count'])  
)  
  
print(channel_sales)
```

Overall Assessment

This is a well-structured transactional sales dataset suitable for analytics, dashboarding, machine learning, and educational purposes. It combines:

- Temporal data
- Geographic data
- Product hierarchy data
- Customer segmentation
- Revenue metrics

The dataset is especially useful for:

- Retail analytics practice
- Business intelligence demonstrations
- Data cleaning exercises
- SQL and Python portfolio projects
- Power BI/Tableau dashboards
- Introductory forecasting workflows